

BNR Licensing & Compliance Portal

1. How the system is arranged

The frontend is in the folder frontend/. I used React, TypeScript, and Vite. It speaks to the backend only through REST. The login token that is the JWT is stored in the browser localStorage. The screen shows buttons for each role, but the screen is not the boss that makes decisions. If someone sends requests without the UI, the backend still checks every rule.

The backend is one Spring Boot application in backend/. All API routes start with /api/. I did not split the system into many small microservices for this project. One reason is time constraints. Another reason is that it is some serious work: when you update one licence case and write an audit line, it is easier if both actions sit in one database transaction inside one running service. I learned in class(Data Intensive Scalable Systems by Paulo Merson at CMU Africa) that microservices have benefits, but they also add work for networks and consistency. For a first version, a modular monolith (one deployable, clear packages inside) was enough.

PostgreSQL stores the main data: people, applications, file metadata, and audit lines. Real files are not inside the database. I save them in a normal folder on the machine. The setting is portal.files.upload-dir (or the environment variable PORTAL_FILES_UPLOAD_DIR). The database only keeps the path, size, and name. Cloud storage like S3 is a future step.

On the developer machine, the browser uses Vite on port 5173. Vite sends /api calls to Spring on port 8080. In production you would serve the built static files in another way, but the idea is the same: the API stays the gate.

2. Diagrams

I drew diagrams with draw.io. I use a few types: sequence, state, class, and a database-style picture. The table below lists them.

Diagram	Type	What you see there
Diagram 1	Database (ER)	Tables and keys from Flyway
Diagram 2	Components	Main parts and how they connect
Diagram 3	State	Stages of an application
Diagram 4	Sequence	One workflow request and audit

Link for diagrams:
<https://drive.google.com/file/d/1yLxwdbQY99GkdiLnB-LSGqChAZXF0Wr/view?usp=sharing>

Logical ERD PostgreSQL (Flyway `V1\_\_schema.sql`)

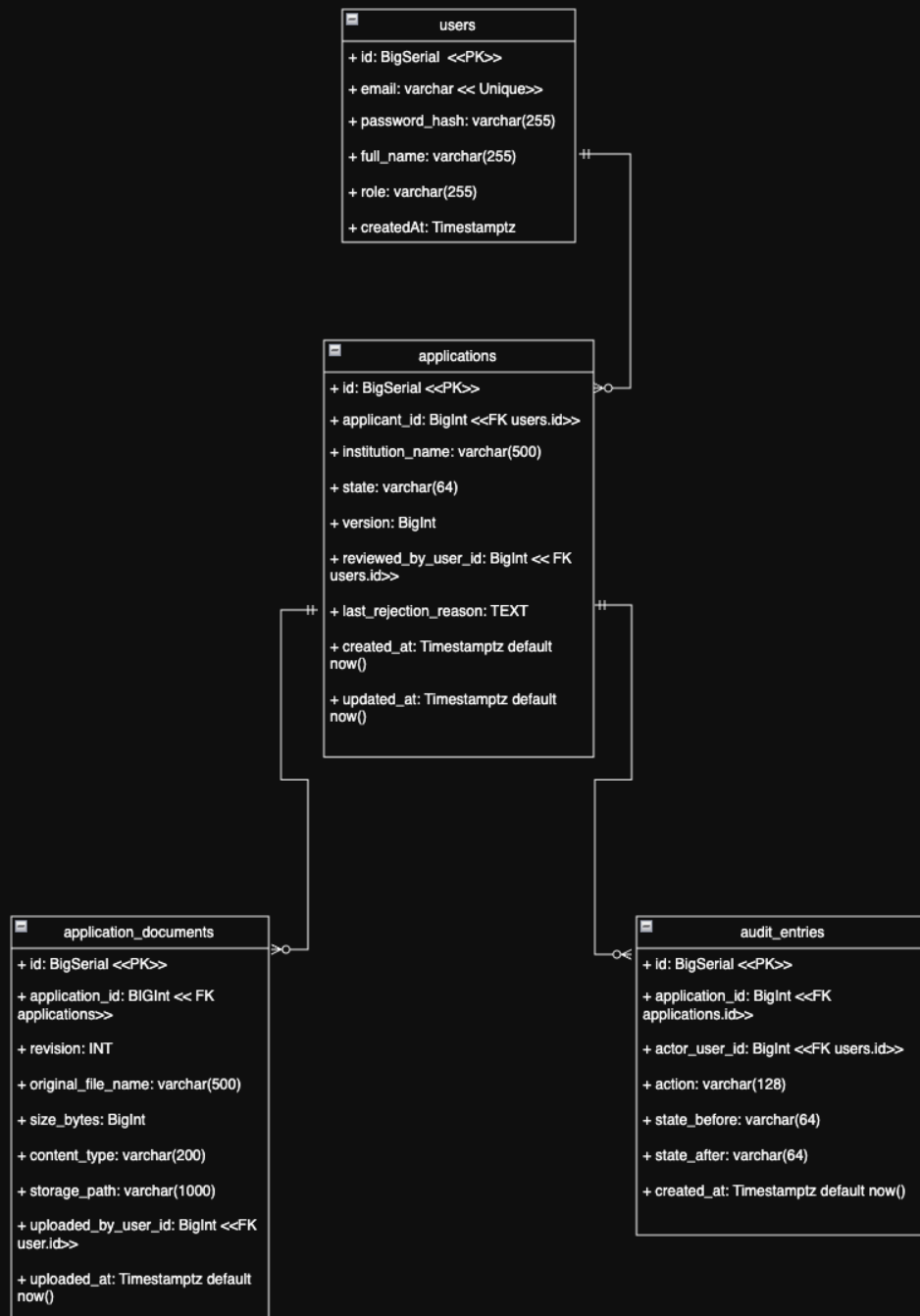


Diagram 1

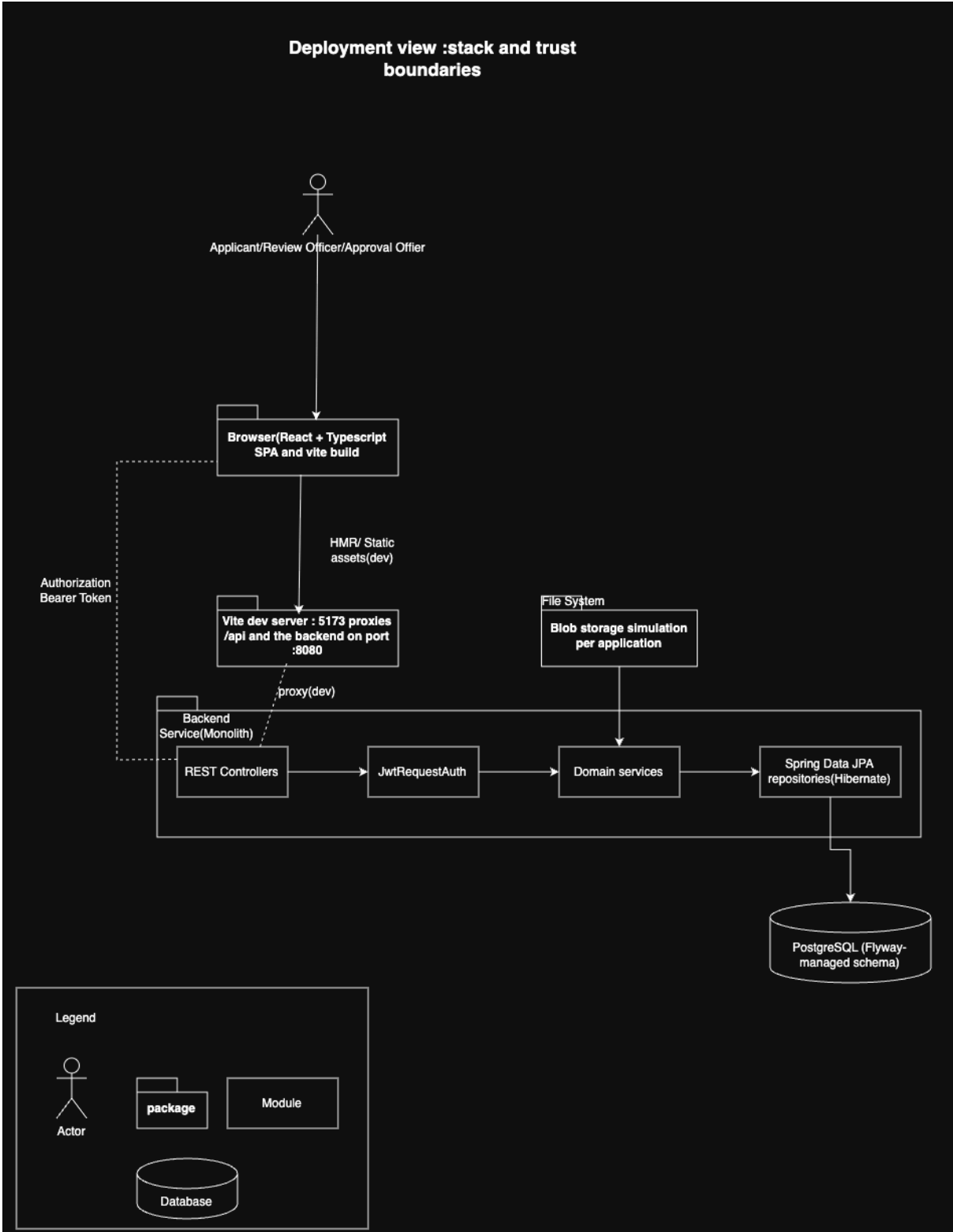


Diagram 2

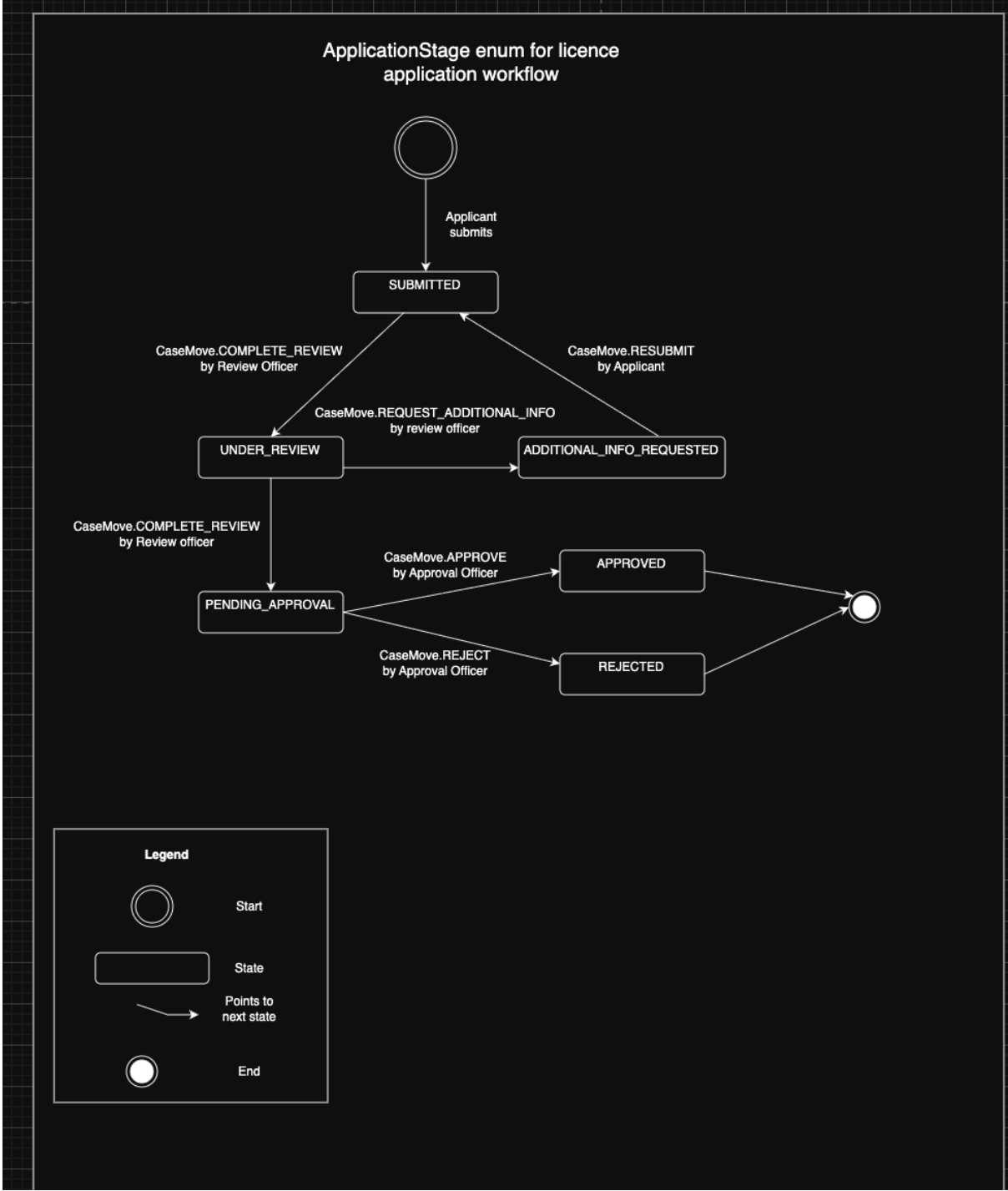


Diagram 3

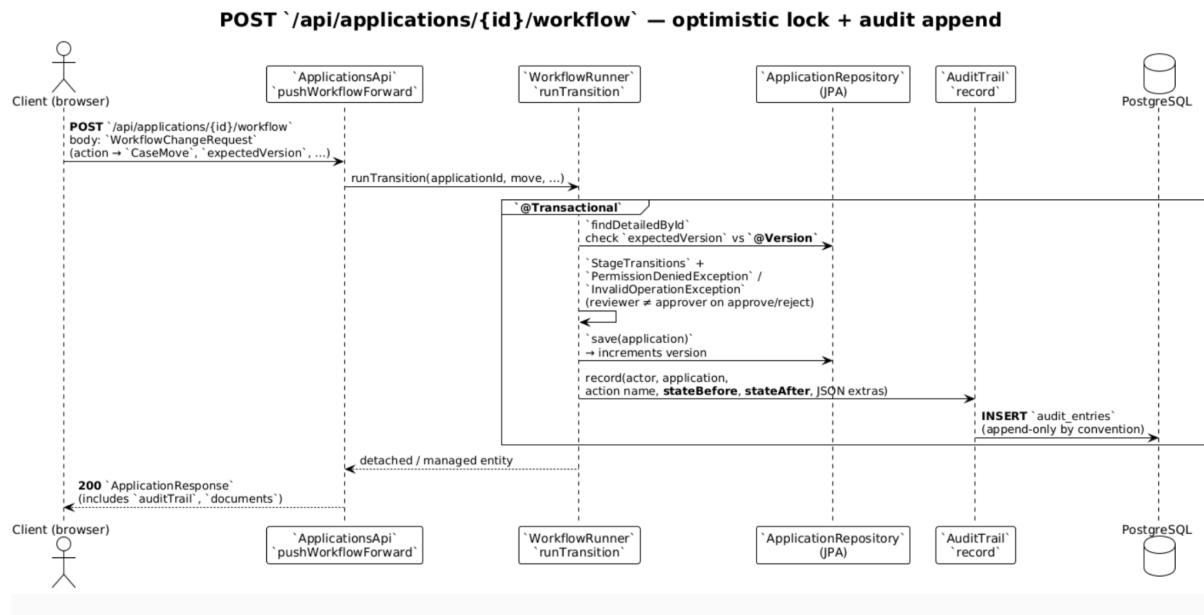


Diagram 4

- The Database in simple words
- **users** — people who sign in. Email, password hash (Bcrypt), full name, and role.
- **applications** — one row per licence request. Institution name, current **state** (stage of work), **version** number for safe updates, link to the applicant, optional link to the officer who finished the review, and optional text if the case was rejected.
- **application_documents** — one row per uploaded file (metadata). It remembers the upload **round** (revision), the file name, size, type, and where the file lives on disk. We keep old rounds so a new upload does not delete history.
- **audit_entries** — a line for each important action. Who did it, what changed (state before and after), time, and extra JSON if needed.

One application has one applicant. It may have one “reviewed by” person later. Many documents and many audit lines can belong to one application.

The file diagrams/database-schema.puml follows the same idea as the Flyway script. Only **application_documents** uses **ON DELETE CASCADE** to the application. Other foreign keys use the normal PostgreSQL rule (no cascade) unless the script says something else.

In code, stages are the enum `ApplicationStage`. Actions are the enum `CaseMove`. The class `StageTransitions` lists which action is allowed from which stage.

The stages are: `SUBMITTED`, `UNDER_REVIEW`, `ADDITIONAL_INFO_REQUESTED`, `PENDING_APPROVAL`, and at the end either `APPROVED` or `REJECTED` (no move out after that).

From stage	Action	Who	Next stage
SUBMITTED	START_REVIEW	Review officer	UNDER_REVIEW

From stage	Action	Who	Next stage
UNDER_REVIEW	REQUEST_ADDITIONAL_INFO	Review officer	ADDITIONAL_INFO_REQUESTED
ADDITIONAL_INFO_REQUESTED	RESUBMIT	Applicant (owner of the case)	SUBMITTED
UNDER_REVIEW	COMPLETE_REVIEW	Review officer	PENDING_APPROVAL (and the system stores who reviewed)
PENDING_APPROVAL	APPROVE	Approval officer (not the same person as the reviewer)	APPROVED
PENDING_APPROVAL	REJECT (with reason)	Approval officer (not the same person as the reviewer)	REJECTED

When a **new** application is created, we do not use a CaseMove. The program sets the stage to SUBMITTED and writes an audit line APPLICATION_CREATED.

If the action is not allowed, WorkflowRunner stops it before the database is updated in a wrong way.

4. The three roles

Applicant can open new applications for themselves, upload files when the stage allows, and resubmit when the office asked for more information. Cannot open other people's cases. Our API returns **403** if they try.

Review officer can list cases, start review, ask for more information, and move the case to "pending approval" after review. Cannot give the final approve or reject.

Approval officer can approve or reject only when the stage is PENDING_APPROVAL. The document that has the write up said clearly: the person who reviewed the file cannot be the same person who gives the final decision. I check that in the backend using reviewed_by_user_id.

5. How I answered the main technical rules

Login and security token: I use JWT. The SPA sends Authorization: Bearer Spring Security checks the token. I still load the user from the database so we do not rely only on old data inside the token.

Rights (authorisation): Service classes check the role again: WorkflowRunner, ApplicationRecords, DocumentUploads. So even if the front page is wrong, the API can say no.

Reviewer and approver must differ: Before APPROVE or REJECT, we compare the current user id with reviewed_by_user_id. If they are the same, we return **403**.

Workflow and conflicts: Allowed moves come from StageTransitions. The column version on applications is for optimistic locking. If two people submit at the same time with the same old version, one request wins and the other gets **409**. There is a test for that case.

Audit trail: We append rows to audit_entries through AuditTrail.record. We do not add code paths that update or delete audit rows in normal work. For a stronger production system, the database administrator could remove UPDATE/DELETE rights on that table.

Documents: Maximum size **5 MB** per file. We set it in Spring and we check again in DocumentUploads.

API errors: We return a small JSON object with error and message. We do not send stack traces to the client. **401** if not logged in. **403** if logged in but not allowed. **400** for bad input or illegal step. **409** for version conflict.

API documentation: OpenAPI JSON is at /v3/api-docs (Springdoc). You can import it into Postman.

Frontend: Buttons appear only for actions that role can do. We show loading and errors so the user is not left with a silent blank page.

6. Future Work

I could split out a small service for email or notifications. I could use a message queue and an outbox pattern as we studied(in Software Architectures class by Swarna) for reliable delivery.

I could harden audit at database level (permissions, or hashing).

I could store files in cloud object storage and run virus checks.

I could run automated tests against Postgres in Docker (Testcontainers) so CI matches production DDL.

I could add paging and search on the API list endpoints.